



IT214 Database Management System

Lab – 9: DB_Project_Assignment_6- Normalization and Schema Refinement

Aastha Shetty	202001260
Hemangi Joshi	202001273

Group: 5
Section: 6
Group Id: 6.13
Date: 11/11/2022

i. List all the Relations & Schemas with all details (Original Design of Database)

- Owner (owner_id, owner_name)
- Manager (manager_id, manager_name)
- Developer (developer_id, developer_name)
- User_input (sr_no, type_of_info, info_description)
- Player (player_id, first_name, last_name, DOB, gender, height, weight)
- Interview (player_id, interview_links)
- Player_country (player_id, player_country_name, player_country_id, noc)
- Sports (player_id, sports_name, location, year)
- Organising_country(event_id, organising_country_id, organising_country_name)
- Events(event_id, event_name, year, controversies, season, year)
- Ceremony (event_id, year, description)
- Chief_guests (event_id, guest_id, guest_first_name, guest_last_name, guest_age, guest_gender, guest_country)
- Rewards (reward_id, std_reward, player_id)
- Medals (medal_id, medal_name, player_id)
- Winner_info (player_id, sports_name, medal_id, reward_id)

ii. Identify and list all types of dependencies (PK, FK, Functional Dependencies) for each relation

1. Owner

PK-Owner_id

FD-(Owner_id $\rightarrow$ owner_name)

2. Manager

PK-manager_id

FD-(manager_id $\rightarrow$ manager_name)

3. Developer

PK-Developer_id

FD-(Developer_id → developer_name)

4. User Input

PK- sr_no

FD-(sr_no → type_of_info, sr_no → info_description)

5. Player

PK - player_id

FD - (player_id→first_name, player_id→last_name, player_id→dob,
player_id→age, player_id→gender,player_id→height, player_id→weight,
dob→age)

6. Interview

PK - player_id

FK - player_id

FD - (player_id→interview_links)

7. Player_country

PK - player_id

FK - player_id

FD - (player_id→player_country_id, player_id→player_country_name,
player_id→noc, player_country_id→player_country_name,
player_country_id→noc)

8. Sports

PK - player_id

FK - player_id

FD - (player_id→sports_name, player_id→location, player_id→year)

9. Organizing_country

PK - event_id

FK - event_id

FD - (event_id→organizing_country_id, event_id→organizing_country_name,
organizing_country_id→organizing_country_name)

10. Events

PK - event_id

FD - (event_id→event_name, event_id→event_name, event_id→season,
event_id→year)

11. Chief Guests

PK - guest_id

FK - event_id

FD - (guest_id→guest_first_name, guest_id→guest_last_name,
guest_id→guest_age, guest_id→guest_gender, guest_id→guest_country,
guest_id→event_id)

12. Ceremony

PK - event_id

FK - event_id

FD - (event_id→year, event_id→description)

13. Rewards

PK - reward_id

FK - player_id

FD - (reward_id→std_reward, reward_id→player_id)

14. Winner_info

PK - player_id

FK - player_id, medal_id, reward_id

FD - (player_id→sports_name, player_id→medal_id, player_id→reward_id)

15. Medals

PK - medal_id

FK - player_id

FD - (medal_id→medal_name)

iii. Redundancy and Analysis

→ **Owner** (owner_id, owner_name)

- There can be the same name associated with different owner_id.
- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **Manager** (manager_id, manager_name)

- There can be the same name associated with different manager_id.
- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **Developer** (developer_id, developer_name)

- There can be the same name associated with different developer_id.
- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **User_input** (sr_no, type_of_info, info_description)

- There can be the same type of info for different info descriptions.
- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **Player** (player_id, first_name, last_name, DOB, age(), gender, height, weight)

- Redundancy: Here we have transitive dependency as player_id→dob, dob→age implies player_id→age).
- We have removed dob so there are no transitive dependencies.
- There are no anomalies in this entity.

→ **Interview** (player_id, interview_links)

- There are no anomalies in this entity
- There are no transitive dependencies.

→ **Player_country** (player_id, player_country_name, player_country_id, noc)

- There can be more than one player associated with the same country id and country name.
- Here there is transitive dependency as ($\text{player_id} \rightarrow \text{player_country_id}$, $\text{player_country_id} \rightarrow \text{player_country_name}$ this implies $\text{player_id} \rightarrow \text{player_country_name}$).
- There are no anomalies in this entity

→ **Sports** (player_id, sports_name, location, year)

- There can be the same sports name associated with different player_id. Same locations can have different years. Same sports_name can also have different locations and years.
- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **Organising_country** (event_id, organising_country_id, organising_country_name)

- Here there is transitive dependency as ($\text{event_id} \rightarrow \text{organizing_country_id}$, $\text{organizing_country_id} \rightarrow \text{organizing_country_name}$ implies $\text{event_id} \rightarrow \text{organizing_country_name}$).
- So we will break into two tables with

Table 1: Attributes: organizing_country_id, organizing_country_name
 $\text{Organizing_country_id} \rightarrow \text{organizing_country_name}$.

Table 2: Attributes: event_id, organizing_country_id
 $\text{Event_id} \rightarrow \text{organizing_country_id}$

- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **Events**(event_id, event_name, year, controversies, season, year)

- One season can have more than one events.
- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **Ceremony** (event_id, year, description)

- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **Chief_guests** (event_id, guest_id, guest_first_name, guest_last_name, guest_age, guest_gender, guest_country)

- Same chief guest can be called for more than one events
- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **Rewards** (reward_id, std_reward, player_id)

- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **Medals** (medal_id, medal_name, player_id)

- There are no anomalies in this entity.
- There are no transitive dependencies.

→ **Winner_info** (player_id, sports_name, medal_id, reward_id)

- There are no anomalies in this entity.
- There are no transitive dependencies.

iv. Normalize the database

There are no composite or multivalued attributes so the condition of atomicity is satisfied. Hence the schema is in 1 NF.

→ **Owner** (owner_id, owner_name)

- There is only one candidate key which is owner_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Manager** (manager_id, manager_name)

- There is only one candidate key which is manager_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Developer** (developer_id, developer_name)

- There is only one candidate key which is developer_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **User_input** (sr_no, type_of_info, info_description)

- There is only one candidate key which is sr_no hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Player** (player_id, first_name, last_name, DOB, age(), gender, height, weight)

- There is only one candidate key which is player_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Interview** (player_id, interview_links)

- There is only one candidate key which is player_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Player_country** (player_id, player_country_name, player_country_id, noc)

- There is only one candidate key which is player_id hence there are no partial dependencies. Therefore it is in 2NF.
- We break the table into two as there is transitive dependency as
Table 1: Attributes: player_country_id, player_country_name, noc
player_country_id → noc player-country_name.
Table 2: Attributes: player_id, player_country_id
player_id → player_country_id.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Sports** (player_id, sports_name, location, year)

- There is only one candidate key which is player_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Organising_country**(event_id, organising_country_id, organising_country_name)

- There is only one candidate key which is event_id hence there are no partial dependencies. Therefore it is in 2NF.
- We break the table into two as there is transitive dependency as

Table 1: Attributes: organizing_country_id, organizing_country_name

Organizing_country_id → organizing_country_name.

Table 2: Attributes: event_id, organizing_country_id

Event_id → organizing_country_id

- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Events**(event_id, event_name, year, controversies, season, year)

- There is only one candidate key which is event_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Ceremony** (event_id, year, description)

- There is only one candidate key which is event_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Chief_guests** (event_id, guest_id, guest_first_name, guest_last_name, guest_age, guest_gender, guest_country)

- There is only one candidate key which is guest_id hence there are no partial dependencies. Therefore it is in 2NF.

- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Rewards** (reward_id, std_reward, player_id)

- There is only one candidate key which is reward_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

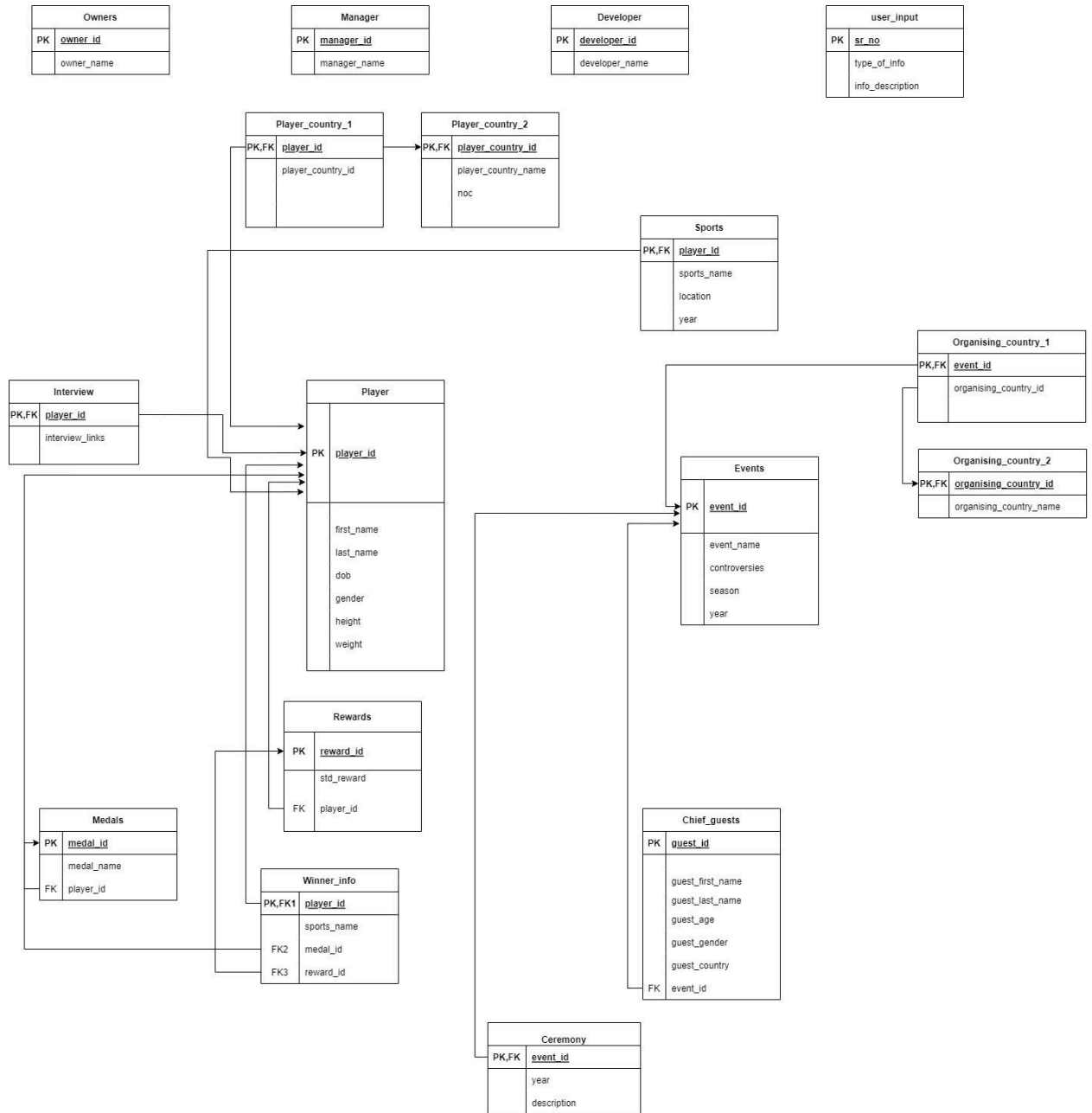
→ **Medals** (medal_id, medal_name, player_id)

- There is only one candidate key which is medal_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

→ **Winner_info** (player_id, sports_name, medal_id, reward_id)

- There is only one candidate key which is player_id hence there are no partial dependencies. Therefore it is in 2NF.
- The functional dependency has candidate key on the left side so it is in BCNF which implies that it is also in 3NF.

ER Relational Model after Normalization:



Relations after Normalization:

- Owner (owner_id, owner_name)

- Manager (manager_id, manager_name)
- Developer (developer_id, developer_name)
- User_input (sr_no, type_of_info, info_description)
- Player (player_id, first_name, last_name, DOB, gender, height, weight)
- Interview (player_id, interview_links)
- Player_country_1 (player_id, player_country_name, noc)
- Player_country_2 (player_id, player_country_id)
- Sports (player_id, sports_name, location, year)
- Organising_country_1(organising_country_id, organising_country_name)
- Organising_country_2(event_id,organising_country_id)
- Events(event_id, event_name, year, controversies, season, year)
- Ceremony (event_id, year, description)
- Chief_guests (event_id,guest_id, guest_first_name, guest_last_name, guest_age, guest_gender, guest_country)
- Rewards (reward_id, std_reward, player_id)
- Medals (medal_id, medal_name,player_id)
- Winner_info (player_id, sports_name, medal_id, reward_id)

3. DDL SCRIPTS

```
CREATE SCHEMA olympics_db;
SET SEARCH_PATH TO olympics_db;
```

1. Player

```
CREATE TABLE IF NOT EXISTS olympics_db.Player
```

```
(
    player_id integer NOT NULL,
    first_name character varying COLLATE pg_catalog."default",
    last_name character varying COLLATE pg_catalog."default",
    gender character varying COLLATE pg_catalog."default",
    height integer NOT NULL,
    weight integer NOT NULL,
    dob character varying COLLATE pg_catalog."default",
    CONSTRAINT users_pkey PRIMARY KEY (player_id)
)
```

```
TABLESPACE pg_default;
```

```
ALTER TABLE IF EXISTS olympics_db.Player  
    OWNER to postgres;
```

2. Owner

```
CREATE TABLE IF NOT EXISTS olympics_db.Owner  
(  
    owner_id integer NOT NULL,  
    owner_name character varying COLLATE pg_catalog."default",  
    PRIMARY KEY (owner_id)  
)
```

```
TABLESPACE pg_default;
```

```
ALTER TABLE IF EXISTS olympics_db.Owner  
    OWNER to postgres;
```

3. Manager

```
CREATE TABLE IF NOT EXISTS olympics_db.Manager  
(  
    manager_id integer NOT NULL,  
    manager_name character varying COLLATE pg_catalog."default",  
    PRIMARY KEY (manager_id)  
)
```

```
TABLESPACE pg_default;
```

```
ALTER TABLE IF EXISTS olympics_db.Manager  
    OWNER to postgres;
```

4. Developer

```
CREATE TABLE IF NOT EXISTS olympics_db.Developer  
(  
    developer_id integer NOT NULL,  
    developer_name character varying COLLATE pg_catalog."default",  
    PRIMARY KEY (developer_id)  
)
```

```
TABLESPACE pg_default;
```

```
ALTER TABLE IF EXISTS olympics_db.Developer  
OWNER to postgres;
```

Description: developer_id has to be unique and not null for each manager to identify them individually.

5. user_input

```
CREATE TABLE IF NOT EXISTS olympics_db.user_input  
(  
    sr_no integer NOT NULL,  
    type_of_info character varying COLLATE pg_catalog."default",  
    info_description character varying COLLATE pg_catalog."default",  
    PRIMARY KEY (sr_no)  
)
```

```
TABLESPACE pg_default;
```

```
ALTER TABLE IF EXISTS olympics_db.user_input  
OWNER to postgres;
```

6. Interview

```
CREATE TABLE IF NOT EXISTS olympics_db.Interview  
(  
    player_id integer,  
    interview_links character varying COLLATE pg_catalog."default",  
    PRIMARY KEY (player_id),  
    FOREIGN KEY (player_id)  
    REFERENCES olympics_db.Player (player_id) MATCH SIMPLE  
    ON UPDATE NO ACTION  
    ON DELETE NO ACTION  
)
```

```
TABLESPACE pg_default;
```

```
ALTER TABLE IF EXISTS olympics_db.Interview  
OWNER to postgres;
```

7. Player_country_1

```
SET SEARCH_PATH TO olympics_db
CREATE TABLE IF NOT EXISTS olympics_db.Player_country_1
(
    player_country_id integer NOT NULL,
    player_country_name character varying COLLATE pg_catalog."default",
    noc character varying COLLATE pg_catalog."default",
    PRIMARY KEY (player_country_id)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS olympics_db.Player_country_1
    OWNER to postgres;
```

8. Player_country_2

```
SET SEARCH_PATH TO olympics_db
CREATE TABLE IF NOT EXISTS olympics_db.Player_country_2
(
    player_id integer NOT NULL,
    player_country_id character varying COLLATE pg_catalog."default",
    PRIMARY KEY (player_id),
    FOREIGN KEY (player_id)
    REFERENCES olympics_db.Player (player_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
)

TABLESPACE pg_default;
```

9. Sports

```
CREATE TABLE IF NOT EXISTS olympics_db.Sports
(
```

```

        player_id integer,
sports_name character varying COLLATE pg_catalog."default",
        location character varying COLLATE pg_catalog."default",
        year integer NOT NULL,
PRIMARY KEY (player_id),
FOREIGN KEY (player_id)
REFERENCES olympics_db.Player (player_id) MATCH SIMPLE
ON UPDATE NO ACTION
ON DELETE NO ACTION
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS olympics_db.Sports
OWNER to postgres;

```

10. Events

```

CREATE TABLE IF NOT EXISTS olympics_db.Events
(
    event_id integer NOT NULL,
    event_name character varying COLLATE pg_catalog."default",
    controversies character varying COLLATE pg_catalog."default",
    season character varying COLLATE pg_catalog."default",
    year integer NOT NULL,
PRIMARY KEY (event_id)
)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS olympics_db.Events
OWNER to postgres;

```

11.Organising_country_1

```

CREATE TABLE IF NOT EXISTS olympics_db.Organising_country_1
(
    organising_country_id integer NOT NULL,
    organising_country_name character varying COLLATE pg_catalog."default",
PRIMARY KEY (organising_country_id)

```

)

TABLESPACE pg_default;

ALTER TABLE IF EXISTS olympics_db.Organising_country_1
OWNER to postgres;

12. Organising_country_2

```
CREATE TABLE IF NOT EXISTS olympics_db.Organising_country_2
(
    event_id integer,
    organising_country_id integer NOT NULL,
    PRIMARY KEY (event_id),
    FOREIGN KEY (organising_country_id)
    REFERENCES olympics_db.Organising_country_1 (organising_country_id)
    MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
)
```

TABLESPACE pg_default;

ALTER TABLE IF EXISTS olympics_db.Organising_country_2
OWNER to postgres;

13. Ceremony

```
CREATE TABLE IF NOT EXISTS olympics_db.Ceremony
(
    event_id integer,
    year integer NOT NULL,
    description character varying COLLATE pg_catalog."default",
    PRIMARY KEY (event_id),
    FOREIGN KEY (event_id)
    REFERENCES olympics_db.Events (event_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
)
```

TABLESPACE pg_default;

ALTER TABLE IF EXISTS olympics_db.Ceremony

OWNER to postgres;

14. Chief_guests

```
CREATE TABLE IF NOT EXISTS olympics_db.Chief_guests
(
    event_id integer,
    guest_id integer NOT NULL,
    guest_first_name character varying COLLATE pg_catalog."default",
    guest_last_name character varying COLLATE pg_catalog."default",
    guest_age integer NOT NULL,
    guest_gender character varying COLLATE pg_catalog."default",
    guest_country character varying COLLATE pg_catalog."default",
    PRIMARY KEY (guest_id),
    FOREIGN KEY (event_id)
    REFERENCES olympics_db.Events (event_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
)
```

TABLESPACE pg_default;

```
ALTER TABLE IF EXISTS olympics_db.Chief_guests
    OWNER to postgres;
```

15. Medals

```
CREATE TABLE IF NOT EXISTS olympics_db.Medals
(
    medal_id integer NOT NULL,
    medal_name character varying COLLATE pg_catalog."default",
    player_id integer,
    year bigint NOT NULL,
    PRIMARY KEY (medal_id),
    FOREIGN KEY (player_id)
    REFERENCES olympics_db.Player(player_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
)
```

TABLESPACE pg_default;

```
ALTER TABLE IF EXISTS olympics_db.Medals
```

OWNER to postgres;

16. Rewards

```
CREATE TABLE IF NOT EXISTS olympics_db.Rewards
(
    reward_id integer NOT NULL,
    player_id integer,
    std_reward character varying COLLATE pg_catalog."default",
    PRIMARY KEY (reward_id),
    FOREIGN KEY (player_id)
    REFERENCES olympics_db.Player(player_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
)
```

TABLESPACE pg_default;

```
ALTER TABLE IF EXISTS olympics_db.Rewards
    OWNER to postgres;
```

17. Winner_info

```
CREATE TABLE IF NOT EXISTS olympics_db.Winner_info
(
    player_id integer,
    sports_name character varying COLLATE pg_catalog."default",
    medal_id integer,
    reward_id integer,
    FOREIGN KEY (player_id)
    REFERENCES olympics_db.Player(player_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION,
    FOREIGN KEY (medal_id)
    REFERENCES olympics_db.Medals(medal_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION,
    FOREIGN KEY (reward_id)
    REFERENCES olympics_db.Rewards(reward_id) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION
)
```

```
TABLESPACE pg_default;
```

```
ALTER TABLE IF EXISTS olympics_db.Winner_info  
OWNER to postgres;
```

Snapshots of tables

1. Owner

Query Query History

1

2

SELECT * FROM olympics_db.owner
ORDER BY owner_id ASC

Data output Messages Notifications



	owner_id [PK] integer 	owner_name character varying 
1	1	Collette
2	2	Aylmer
3	3	Sarita
4	4	Ali

Total rows: 4 of 4 Query complete 00:00:00.125

2. Developer

Query

Query History

1

SELECT * FROM olympics_db.developer

2

ORDER BY developer_id ASC

Data output

Messages

Notifications

developer_id

[PK] integer

developer_name

character varying

1

1

Hugibert

2

2

Colan

3

3

Keary

4

4

Sarette

5

5

Lew

6

6

Charlotta

7

7

Caterina

8

8

Garv

Total rows: 8 of 8

Query complete 00:00:00.185

3. Manager

Query Query History

```
1 SELECT * FROM olympics_db.manager
2 ORDER BY manager_id ASC
```

Data output Messages Notifications



	manager_id [PK] integer	manager_name character varying
1	1	Ricard Haversum
2	2	Cass Stroder
3	3	Benjamin Tripco...
4	4	Natty Annion
5	5	Ferrell Rablin
6	6	Crystal Fawssett
7	7	Odille Schrinel
8	8	Quincey Mowne

Total rows: 8 of 8 Query complete 00:00:00.192

4. User_input

Query

Query History

1

SELECT * FROM olympics_db.user_input

2

ORDER BY sr_no ASC

Data output

Messages

Notifications

sr_no

[PK] integer

type_of_info

character varying

info_description

character varying

1

1

Morbi porttitor lo...

Etiam pretium ia...

2

2

Nulla suscipit lig...

Proin leo odio, po...

3

3

Integer pede just...

Morbi non quam ...

4

4

Suspendisse pot...

Nulla suscipit lig...

5

5

Proin risus.

Etiam vel augue. ...

6

6

Nulla tellus.

Vestibulum ante i...

7

7

Aliquam sit amet...

Sed accumsan fe...

8

8

Sed accumsan fe...

Integer tincidunt ...

9

9

In congue.

In quis justo. Ma...

10

10

Maecenas rhonc...

Donec quis orci e...

11

11

Vestibulum qua...

Maecenas rhonc...

12

12

Nunc nisl.

Suspendisse acc...

13

13

Ut tellus.

Donec ut mauris ...

14

14

Phasellus sit am...

Curabitur at ipsu...

15

15

Donec ut dolor.

Morbi quis tortor ...

16

16

Donec odio justo,...

Integer ac neque....

17

17

Proin leo odio, po...

Nam nulla. Integ...

Total rows: 20 of 20

Query complete 00:00:00.208

5. Player

Query

Query History

```

1 SELECT * FROM olympics_db.player
2 ORDER BY player_id ASC

```

Data output

Messages

Notifications

≡

📄

▼

📋

🗑️

🗄️

⬇️

📈

	player_id [PK] integer	first_name character varying	last_name character varying	gender character varying	height integer	weight integer	dob character varying
1	1	Stearn	Grinnov	M	179	78	28-10-1965
2	2	Berny	Tomankiewicz	F	169	82	19-12-1950
3	3	Grier	Clooney	F	261	110	05-10-2000
4	4	Hildagarde	Cuttle	F	101	77	07-04-1977
5	5	Lillian	Tippings	F	16	87	31-05-2026
6	6	Anneliese	Prince	F	98	192	10-06-1934
7	7	Agnola	Halvosen	F	265	9	16-10-1952
8	8	Grace	Colhoun	F	59	31	15-04-2003
9	9	Bertine	Tapper	F	79	114	30-06-2020
10	10	Sumner	Temple	M	67	183	01-05-2012
11	11	Verla	Luesley	F	146	35	13-10-1939
12	12	Orren	Grattan	M	181	40	12-08-2013
13	13	Bowie	O'Leahy	M	257	32	22-08-2003
14	14	Trenton	Eastup	M	197	62	18-06-1978
15	15	Bernita	Popov	F	35	178	24-05-2016
16	16	Valina	Simak	F	26	84	11-04-1947
17	17	Teddi	Blenkensop	F	221	74	21/4/1899

Total rows: 50 of 50

Query complete 00:00:00.153

6. Interview

Query Query History

```
1 SELECT * FROM olympics_db.interview
2 ORDER BY player_id ASC
```

Data output Messages Notifications



	player_id [PK] integer	interview_links character varying
1	1	http://dummyima...
2	2	http://dummyima...
3	3	http://dummyima...
4	4	http://dummyima...
5	6	http://dummyima...
6	10	http://dummyima...
7	11	http://dummyima...
8	12	http://dummyima...
9	13	http://dummyima...
10	14	http://dummyima...
11	17	http://dummyima...
12	19	http://dummyima...
13	21	http://dummyima...
14	22	http://dummyima...
15	24	http://dummyima...
16	27	http://dummyima...
17	30	http://dummyima...

Total rows: 30 of 30 Query complete 00:00:00.262

7. Player_country_1

Query

Query History

```

1 SELECT * FROM olympics_db.player_country_1
2 ORDER BY player_country_id ASC

```

Data output

Messages

Notifications

≡

📄

▼

📋

🗑️

🗄️

📥

📈

	player_country_id [PK] integer	player_country_name character varying	noc character varying
1	1	Indonesia	vulputate
2	2	Luxembourg	rutrum
3	3	Japan	vel nisl
4	4	Uganda	ligula nec
5	5	China	sit amet
6	6	Colombia	nulla tellus
7	7	Kuwait	sapien
8	8	Paraguay	eu pede
9	9	Philippines	rhoncus aliquam
10	10	Macedonia	justo
11	11	France	nam tristique
12	12	China	sed tincidunt
13	13	Albania	in hac
14	14	China	ornare
15	15	Iran	volutpat
16	16	Portugal	convallis duis

Total rows: 50 of 50

Query complete 00:00:00.156

8. Player_country_2

Query

Query History

1 SELECT * FROM olympics_db.player_country_2

2 ORDER BY player_id ASC

Data output

Messages

Notifications

	player_id [PK] integer	player_country_id character varying
1	1	39
2	2	49
3	3	50
4	4	25
5	5	8
6	6	40
7	7	11
8	8	23
9	9	25
10	10	23
11	11	45
12	12	41
13	13	26
14	14	22
15	15	11
16	16	27

Total rows: 50 of 50

Query complete 00:00:00.340

9. Sports

Query Query History

```
1 SELECT * FROM olympics_db.sports
2 ORDER BY player_id ASC
```

Data output Messages Notifications



	player_id [PK] integer	sports_name character varying	location character varying	year integer
1	1	football	Ireland	2007
2	2	badminton	Poland	2011
3	3	swimming	Italy	1993
4	4	swimming	China	2012
5	5	badminton	Indonesia	2011
6	6	Marathon	Burundi	2011
7	7	football	Venezuela	2004
8	8	swimming	Albania	2001
9	9	Marathon	Pakistan	1995
10	10	football	Indonesia	2010
11	11	Marathon	Cameroon	2004
12	12	badminton	Luxembourg	1989
13	13	Marathon	China	1992
14	14	Marathon	Russia	2011
15	15	swimming	Micronesia	2002
16	16	wrestling	Vietnam	1999

Total rows: 50 of 50 Query complete 00:00:00.267

10. Events

Query

Query History

1

SELECT * FROM olympics_db.events

2

ORDER BY event_id ASC

Data output

Messages

Notifications

	event_id [PK] integer	event_name character varying	controversies character varying	season character varying	year integer
1	1	chaddy0	Fusce consequat...	Paralympic	2005
2	2	zsherebrook1	Mauris enim leo, ...	Winter	2002
3	3	iseagrove2	Quisque id justo ...	Winter	1993
4	4	gsimmon3	Nunc rhoncus du...	Summer	1994
5	5	zpaszek4	Duis aliquam con...	Youth games	2008
6	6	fwoodbridge5	Donec ut mauris ...	Paralympic	2004
7	7	gstorrier6	Nullam porttitor l...	Youth games	1994
8	8	hcopyn7	Suspendisse pot...	Summer	1991
9	9	bbonallack8	Aliquam erat volu...	Winter	2007
10	10	rsebborn9	Morbi non quam ...	Winter	1984
11	11	ditzakovitza	Integer a nibh.	Summer	2004
12	12	graunb	Pellentesque at n...	Youth games	2012
13	13	ybedboroc	Donec quis orci e...	Winter	1990
14	14	marnautd	Mauris sit amet e...	Summer	1987
15	15	kjanuaryste	Aenean auctor gr...	Paralympic	2002
16	16	dmonkhousef	Proin risus. Prae...	Youth games	2012

Total rows: 50 of 50

Query complete 00:00:00.825

11. Organising_country_1

Query Query History

```
1 SELECT * FROM olympics_db.organising_country_1
2 ORDER BY organising_country_id ASC
```

Data output Messages Notifications

       			
	organising_country_id [PK] integer		organising_country_name character varying
1	1		Indonesia
2	2		Cuba
3	3		Russia
4	4		Macedonia
5	5		Bulgaria
6	6		Azerbaijan
7	7		France
8	8		Indonesia
9	9		Argentina
10	10		Myanmar
11	11		Peru
12	12		Indonesia
13	13		Morocco
14	14		Czech Republic
15	15		United States
16	16		France
Total rows: 50 of 50		Query complete 00:00:00.295	

12. Organising_country_2

Query

Query History

1 SELECT * FROM olympics_db.organising_country_2

2 ORDER BY event_id ASC

Data output

Messages

Notifications

<

13. Ceremony

Query Query History

```
1 SELECT * FROM olympics_db.ceremony
2 ORDER BY event_id ASC
```

Data output Messages Notifications



	event_id [PK] integer	year integer	description character varying
1	1	1984	In tempor, turpis ...
2	2	1999	Curabitur grvida...
3	3	2009	Donec diam nequ...
4	4	2002	Sed ante. Vivamu...
5	5	1987	Maecenas ut ma...
6	6	2003	Vivamus vel nulla...
7	7	1990	Duis ac nibh. Fus...
8	8	2007	Nulla tellus. In sa...
9	9	2011	In congue. Etiam ...
10	10	2003	Nunc purus. Pha...
11	11	2006	Donec semper sa...
12	12	1998	In congue. Etiam ...
13	13	2004	Proin at turpis a ...
14	14	2010	Aenean fermentu...
15	15	1998	Suspendisse pot...
16	16	1996	Pellentesque vive...

Total rows: 50 of 50

Query complete 00:00:00.927

14. Chief_guests

Query Query History

```
1 SELECT * FROM olympics_db.chief_guests
2 ORDER BY guest_id ASC
```

Data output Messages Notifications



	event_id integer	guest_id [PK] integer	guest_first_name character varying	guest_last_name character varying	guest_age integer	guest_gender character varying	guest_country character varying
1	36	1	Maëlann	Lurner	90	Female	Serbia
2	50	2	Ruò	Brear	67	Female	Venezuela
3	28	3	Mélia	Kingscot	8	Male	Greece
4	12	4	Maëline	Heinrich	88	Male	United States
5	22	5	Angèle	St. Leger	98	Female	Kyrgyzstan
6	20	6	Michèle	Coade	67	Female	Indonesia
7	7	7	Uò	Mattiussi	37	Male	Ireland
8	35	8	Cécile	Cavaller	48	Male	Philippines
9	2	9	Publicité	Gannicleff	39	Male	Chad
10	4	10	Chloé	Reolfi	26	Male	Palestinian Territ...
11	14	11	Pélagie	Varey	66	Male	Mauritius
12	1	12	Håkan	Lilleycrop	19	Male	Indonesia
13	26	13	Laïla	Hedingham	2	Female	China
14	37	14	Joséphine	Domek	97	Male	Ivory Coast
15	28	15	André	Espine	41	Female	China
16	48	16	Mårten	Tarquini	56	Male	Russia

Total rows: 50 of 50 Query complete 00:00:00.191

15. Medals

Query Query History

```
1 SELECT * FROM olympics_db.medals
2 ORDER BY medal_id ASC
```

Data output Messages Notifications

	medal_id [PK] integer	medal_name character varying	player_id integer	year bigint
1	1	dolor	1	2009
2	2	sed	2	1999
3	3	arcu	3	1985
4	4	non	4	2005
5	5	orci	5	2001
6	6	ante	6	2008
7	7	mauris	7	1992
8	8	aliquet	8	2007
9	9	duis	9	2006
10	10	orci	10	2006
11	11	curabitur	11	2012
12	12	vivamus	12	1993
13	13	ac	13	2007
14	14	non	14	2008
15	15	amet	15	2002
16	16	duis	16	1989
Total rows: 25 of 25		Query complete 00:00:00.185		

16. Rewards

Query

Query History

1

SELECT * FROM olympics_db.rewards

2

ORDER BY reward_id ASC

Data output

Messages

Notifications

	reward_id [PK] integer	player_id integer	std_reward character varying
1	1	1	\$79875.28
2	2	2	\$75650.52
3	3	3	\$85381.96
4	4	4	\$76148.30
5	5	5	\$39983.72
6	6	6	\$75772.23
7	7	7	\$55688.40
8	8	8	\$82201.36
9	9	9	\$15775.81
10	10	10	\$28805.34
11	11	11	\$70371.51
12	12	12	\$16590.76
13	13	13	\$77708.05
14	14	14	\$18763.83
15	15	15	\$38561.53
16	16	16	\$10866.90

Total rows: 25 of 25

Query complete 00:00:00.341

17. Winner_info

Query

Query History

1

SELECT * FROM olympics_db.winner_info

2

Data output

Messages

Notifications

	player_id integer	sports_name character varying	medal_id integer	reward_id integer
1	1	badminton	1	1
2	2	swimming	2	2
3	3	swimming	3	3
4	4	badminton	4	4
5	5	Marathon	5	5
6	6	Marathon	6	6
7	7	badminton	7	7
8	8	Marathon	8	8
9	9	Marathon	9	9
10	10	swimming	10	10
11	11	wrestling	11	11
12	12	football	12	12
13	13	wrestling	13	13
14	14	swimming	14	14
15	15	badminton	15	15
16	16	wrestling	16	16

Total rows: 25 of 25

Query complete 00:00:00.408

SQL Queries

1 . Name of the chief guest having maximum age

```
select guest_first_name, guest_last_name
from olympics_db.chief_guests
where guest_age = (select max(guest_age)) from
olympics_db.chief_guests);
```

Query Query History

```
1 --1. Name of the chief guest having maximum age
2 select guest_first_name, guest_last_name
3 from olympics_db.chief_guests
4 where guest_age = (select max(guest_age) from olympics_db.chief_guests);
```

Data output Messages Notifications

	guest_first_name character varying	guest_last_name character varying
1	Angèle	St. Leger

Total rows: 1 of 1 Query complete 00:00:00.136

2. Find the number of male and female players.

```
Select count(*), "gender" From olympics_db.player  
Group by "gender";
```

```
6  --2. Find the number of male and female players.  
7  Select count(*), "gender" From olympics_db.player  
8  Group by "gender";  
9  
10  
11  
12  
13  
14
```

Data output

Messages

Notifications



	count bigint	gender character varying
1	22	M
2	28	F

3. Find guest id and name whose age is less than avg guest age.

```
select guest_id, guest_first_name, guest_last_name  
from olympics_db.chief_guests g1  
where g1."guest_age" < (select avg(guest_age) from  
olympics_db.chief_guests);
```

Query Query History

```

9
10 --3. Find guest id and name whose age is less than avg guest age.
11 select guest_id, guest_first_name, guest_last_name
12 from olympics_db.chief_guests g1
13 where g1."guest_age" < (select avg(guest_age) from olympics_db.chief_guests);
14

```

Data output Messages Notifications

	guest_id [PK] integer	guest_first_name character varying	guest_last_name character varying
1	3	Mélia	Kingscote
2	7	Uò	Mattiussi
3	8	Cécile	Cavalier
4	9	Publicité	Gannicleff
5	10	Chloé	Reolfi
6	12	Håkan	Lilleycrop
7	13	Laila	Hedingham
8	15	André	Espine
9	17	Gwenaëlle	Collinwood
10	18	Rébecca	Blondin
11	21	Maiwenn	McGiveen
12	23	Åslög	Gerrie
13	29	Wá	Keese
14	30	Åke	Shawley
15	31	Meðla	Appleford

Total rows: 22 of 22
Query complete 00:00:00.091

4. Find list of players having weight less than 90 and greater than 40.

```
SELECT player_id, first_name, weight
from olympics_db.player
where weight>=(select weight from olympics_db.player where weight=40)
and
weight<=(select weight from olympics_db.player where weight=90);
```

QueryQuery History

14

15 --4. Find list of players having weight less than 90 and greater than 40.
16 SELECT player_id, first_name, weight
17 from olympics_db.player
18 where weight>=(select weight from olympics_db.player where weight=40) and
19 weight<=(select weight from olympics_db.player where weight=90);
20
21 --5. List the sports name that are played in summer olympics. error

Data outputMessagesNotifications

≡+

📄

▼

📋

🗑️

🗄️

⬇️

📈

	player_id [PK] integer	first_name character varying	weight integer
1	1	Stearn	78
2	2	Berny	82
3	4	Hildagarde	77
4	5	Lillian	87
5	12	Orren	40
6	14	Trenton	62
7	16	Valina	84
8	17	Teddi	74
9	22	Fayina	90
10	23	Jojo	60
11	24	Arel	68
12	25	Fitz	77
13	29	Wini	50
14	27	Conchita	72

Total rows: 18 of 18

Query complete 00:00:00.064

5. List the sports names that are played in summer olympics.

```
SELECT *
from olympics_db.events
where season='Summer';
```

Query

Query History

20

21 --5. List the sports name that are played in summer olympics.

22 SELECT *

23 from olympics_db.events

24 where season='Summer';

25

26 --6. List the count of events held in different olympics.

Data output

Messages

Notifications

+

📄

▼

📋

🗑️

🗄️

⬇️

📈

	event_id [PK] integer	event_name character varying	controversies character varying	season character varying	year integer
1	4	gsimmon3	Nunc rhoncus du...	Summer	1994
2	33	nenderwickw	Suspendisse acc...	Summer	2011
3	8	hcopyn7	Suspendisse pot...	Summer	1991
4	11	ditzakovitza	Integer a nibh.	Summer	2004
5	14	marnautd	Mauris sit amet e...	Summer	1987
6	18	mkunkelh	Praesent blandit. ...	Summer	2010
7	24	glaugheranen	Suspendisse acc...	Summer	2004
8	26	cstichallp	Cum sociis natoq...	Summer	2000
9	31	gkobierieckiu	Suspendisse pot...	Summer	1985
10	41	lpayze14	Proin leo odio, po...	Summer	1992

Total rows: 10 of 10

Query complete 00:00:08.767

6. List the count of events held in different seasons olympics.

```

Select count(*),"season"
from olympics_db.events
Group by "season";

```

```

20
21 --6. List the count of events held in different olympics.
22 Select count(*),"season" from olympics_db.events
23 Group by "season";
24
25
26
27

```

Data output Messages Notifications

	count bigint	season character varying
1	10	Winter
2	10	Summer
3	1	Youth games
4	17	Youth games
5	1	Winter
6	11	Paralympic

7. Display the interview links of player id<10.

```

select * from olympics_db.interview
where player_id < 10;

```

Query
Query History

```

30 --7. Display the interview links of player id<10.
31 select *
32 from olympics_db.interview
33 where player_id <10
34
35

```

Data output
Messages
Notifications

≡+

📄

▼

📋

🗑️

🗄️

⬇️

📈

	player_id [PK] integer	interview_links character varying
1	3	http://dummyima...
2	2	http://dummyima...
3	4	http://dummyima...
4	6	http://dummyima...
5	1	http://dummyima...

Total rows: 5 of 5
Query complete 00:00:08.527

8. Find all the details of top 5 recently conducted events in olympics.

```

select * from olympics_db.events
ORDER BY "year" desc limit 5

```


QueryQuery History

40

41 --9 List the event_id, description and year of top 7 old events.

42 select * from olympics_db.ceremony

43 ORDER BY "year" asc limit 7

44

45 --10. Print the user input with serial no. greater than 15.

Data outputMessagesNotifications

	player_id [PK] integer	interview_links character varying
1	3	http://dummyima...
2	2	http://dummyima...
3	4	http://dummyima...
4	6	http://dummyima...
5	1	http://dummyima...

Total rows: 5 of 5Query complete 00:00:08.527

10. Print the user input with serial no. greater than 15.

```
SELECT * from olympics_db.user_input
where sr_no>(select sr_no from olympics_db.user_input where sr_no=15)
```

Query Query History

```

44+
45 --10. Print the user input with serial no. greater than 15.
46 SELECT * from olympics_db.user_input
47 where sr_no>(select sr_no from olympics_db.user_input where sr_no=15)
48
49
50 --11. List of Players having reward less than maximum reward

```

Data output Messages Notifications

	sr_no [PK] integer	type_of_info character varying	info_description character varying
1	16	Donec odio justo,...	Integer ac neque....
2	17	Proin leo odio, po...	Nam nulla. Integ...
3	18	Integer a nibh.	Nam nulla. Integ...
4	19	Mauris ullamcorp...	Donec semper sa...
5	20	Donec dapibus.	Pellentesque ultri...

Total rows: 5 of 5 Query complete 00:00:00.123

11. List of Players having reward less than maximum reward

```

select first_name,last_name
from olympics_db.player natural join olympics_db.rewards
where player.player_id=rewards.player_id and std_reward< (select
max(std_reward) from olympics_db.rewards)

```

Query Query History Scratch Pad

```

1 --11. List of Players having reward less than mximum reward
2
3 select first_name,last_name
4 from olympics_db.player natural join olympics_db.rewards
5 where player.player_id=rewards.player_id and std_reward<(select max(std_reward) from olympics_db.r

```

Data output Messages Notifications

	first_name character varying	last_name character varying
1	Stearn	Grinnov
2	Berny	Tomankiewicz
3	Grier	Clooney
4	Hildagarde	Cuttle
5	Lillian	Tippings
6	Anneliese	Prince
7	Agnola	Halvosen
8	Grace	Colhoun
9	Bertine	Tapper
10	Sumner	Temple
11	Verla	Luesley
12	Orren	Grattan
13	Bowie	O'Leahv

Total rows: 24 of 24 Query complete 00:00:00.057 Ln 1,

12. Find the player_id with height greater than 160. Return a temp table with player_id, first_name, last_name and height in the result table. Make sure the records are visible in results.

```

create or replace function olympics_db.temp_player()
returns table (a integer, b character varying ,c character varying, d float)
language "plpgsql"
as $body$
declare rcd record;
begin
drop table if exists olympics_db.temp_table;
create table olympics_db.temp_table(a integer, b character varying ,
                                     c character
varying, d float);
for rcd in (select f1.player_id, f1.first_name, f1.last_name, f1.height
            from olympics_db.player f1 )
loop
if(rcd."height" > 160) then
insert into olympics_db.temp_table values (rcd.player_id, rcd.first_name,
rcd.last_name, rcd.height);
end if;

```

```
end loop;
return query table olympics_db.temp_table;
end;
$body$;
select * from olympics_db.temp_player();
```

```
--12. Find the player_id with height greater than 160. Return a
--temp table with player_id, first_name, last_name and height in the
--result table. Make sure the records are visible in results.
create or replace function olympics_db.temp_player()
returns table (a integer, b character varying ,c character varying, d float)
language "plpgsql"
as $body$
declare rcd record;
begin
drop table if exists olympics_db.temp_table;
create table olympics_db.temp_table(a integer, b character varying ,
                                   c character varying, d float);
for rcd in (select f1.player_id, f1.first_name, f1.last_name, f1.height
            from olympics_db.player f1 )
loop
if(rcd."height" > 160) then
insert into olympics_db.temp_table values (rcd.player_id, rcd.first_name,
                                           rcd.last_name, rcd.height);
end if;
end loop;
return query table olympics_db.temp_table;
end;
$body$;
select * from olympics_db.temp_player();
```

Data output Messages Notifications				
	a integer	b character varying	c character varying	d double precision
10	20	Law	Filipyev	213
11	23	Jojo	Zoren	219
12	24	Arel	Tomaszek	174
13	25	Fitz	Strooband	252
14	26	Adrienne	Pighills	170
15	27	Tillie	Simacek	272
16	30	Dannie	Ciciari	171
17	31	Von	Eltune	198
18	35	Atalanta	Pinnion	217
19	39	Felipe	Gorhardt	257
20	40	Tucker	Czadla	210
21	44	Elbert	Sergean	205
22	46	Dimitri	Tickle	293
23	48	Fayth	Allam	252
24	50	Cassius	Kempster	191

13. Print event id, organizing country id and organizing country name.

```
SELECT p1."event_id",p1."organising_country_id",
h1."organising_country_name"
from olympics_db.organising_country_2 p1 JOIN
olympics_db.organising_country_1 h1
ON p1."organising_country_id" = h1."organising_country_id";
```

Query Query History

```

71 --13. Print event id, organizing country id and organizing country name.
72 SELECT p1."event_id",p1."organising_country_id", h1."organising_country_name"
73 from olympics_db.organising_country_2 p1 JOIN olympics_db.organising_country_1 h1
74 ON p1."organising_country_id" = h1."organising_country_id";
75
76 --14. Print name of guest having most recent appearance in olympics

```

Data output Messages Notifications

	event_id integer	organising_country_id integer	organising_country_name character varying
1	1	45	Saudi Arabia
2	2	43	Indonesia
3	3	21	Russia
4	4	49	Russia
5	5	22	Nigeria
6	6	5	Bulgaria
7	7	18	Tunisia
8	8	31	China
9	9	42	Indonesia
10	10	19	China
11	11	17	Indonesia
12	12	18	Tunisia
13	13	19	China
14	14	11	Peru
15	15	20	China

Total rows: 50 of 50 Query complete 00:00:00.085

14. Print name of guest having most recent appearance in olympics.

```

SELECT event_id, guest_first_name, guest_last_name FROM
olympics_db.chief_guests g1
WHERE g1."event_id" in
(SELECT t1."event_id" FROM olympics_db.events t1
WHERE year = (SELECT MAX(year) FROM olympics_db.events));

```

Query Query History

```

76 --14.Print name of guest having most recent appearance in olympics.
77 SELECT event_id, guest_first_name, guest_last_name FROM olympics_db.chief_guests g1
78 WHERE g1."event_id" in
79 (SELECT t1."event_id" FROM olympics_db.events t1
80 WHERE year = (SELECT MAX(year) FROM olympics_db.events));
81
82 --15 Display the count of the sport won the least by creating a view which

```

Data output Messages Notifications

	event_id integer	organising_country_id integer	organising_country_name character varying
1	1	45	Saudi Arabia
2	2	43	Indonesia
3	3	21	Russia
4	4	49	Russia
5	5	22	Nigeria
6	6	5	Bulgaria
7	7	18	Tunisia
8	8	31	China
9	9	42	Indonesia
10	10	19	China
11	11	17	Indonesia
12	12	18	Tunisia
13	13	19	China
14	14	11	Peru

Total rows: 50 of 50 Query complete 00:00:00.085

15. Display the count of the sport won the least by creating a view which displays the count of times sport is won

```

CREATE VIEW win as
SELECT sports_name,count(sports_name) from
olympics_db.winner_info
group by sports_name
select min(count) from win;

```

Query

Query History

70

--15 Display the count of the sport won the least by creating a view which
---displays the count of times sport is won
CREATE VIEW win as
SELECT sports_name,count(sports_name) from olympics_db.winner_info
group by sports_name
select min(count) from win;

71

72

73

74

75

76

77

78

Data output

Messages

Notifications

≡+

📄

▼

📋

🗑️

🗄️

⬇️

📈

	min	🔒
	bigint	
1	1	

Total rows: 1 of 1

Query complete 00:00:00.046

16. Print player id, medal name and sports name.

```
SELECT p1."player_id",p1."medal_name", h1."sports_name"  
from olympics_db.medals p1 JOIN olympics_db.winner_info h1  
ON p1."player_id" = h1."player_id";
```

Query Query History

```
1 --16. Print player id, medal name and sports name.
2 SELECT p1."player_id",p1."medal_name", h1."sports_name"
3 from olympics_db.medals p1 JOIN olympics_db.winner_info h1 ON p1."player_id" =
4 h1."player_id";
5
```

Data output Messages Notifications

	player_id integer	medal_name character varying	sports_name character varying
1	1	dolor	badminton
2	2	sed	swimming
3	3	arcu	swimming
4	4	non	badminton
5	5	orci	Marathon
6	6	ante	Marathon
7	7	mauris	badminton
8	8	aliquet	Marathon
9	9	duis	Marathon
10	10	orci	swimming
11	11	curabitur	wrestling
12	12	vivamus	football
13	13	ac	wrestling
14	14	non	swimming
15	15	amet	badminton
Total rows: 25 of 25 Query complete 00:00:00.053			

17. Create a view of medals.

```
CREATE OR REPLACE VIEW list as
SELECT * FROM olympics_db.medals
where medal_id <=(select medal_id FROM olympics_db.medals where
medal_id= 10);
SELECT * FROM list
```

QueryQuery History

```

98 --17. Create view of medals.
99 CREATE OR REPLACE VIEW list as
100 SELECT * FROM olympics_db.medals
101 where medal_id <=(select medal_id FROM olympics_db.medals where medal_id= 10);
102 select * from list
103

```

Data outputMessagesNotifications

	medal_id integer	medal_name character varying	player_id integer	year bigint
1	1	dolor	1	2009
2	2	sed	2	1999
3	3	arcu	3	1985
4	4	non	4	2005
5	5	orci	5	2001
6	6	ante	6	2008
7	7	mauris	7	1992
8	8	aliquet	8	2007
9	9	duis	9	2006
10	10	orci	10	2006

Total rows: 10 of 10

Query complete 00:00:05.202

18. Print event id, event name and description.

```

SELECT p1."event_id",p1."event_name", h1."description"
from olympics_db.events p1 JOIN olympics_db.ceremony h1
ON p1."event_id" = h1."event_id";

```

Query Query History

```

5 --18. Print event id, event name and description.
6 SELECT p1."event_id",p1."event_name", h1."description"
7 from olympics_db.events p1 JOIN olympics_db.ceremony h1
8 ON p1."event_id" = h1."event_id";
9

```

Data output Messages Notifications

	event_id integer	event_name character varying	description character varying
1	1	chaddy0	In tempor, turpis ...
2	2	zsherebrook1	Curabitur grvida...
3	3	iseagrove2	Donec diam nequ...
4	4	gsimmon3	Sed ante. Vivamu...
5	5	zpaszek4	Maecenas ut ma...
6	6	fwoodbridge5	Vivamus vel nulla...
7	7	gstorrier6	Duis ac nibh. Fus...
8	8	hcopyn7	Nulla tellus. In sa...
9	9	bbonallack8	In congue. Etiam ...
10	10	rsebborn9	Nunc purus. Pha...
11	11	ditzakovitza	Donec semper sa...
12	12	graunb	In congue. Etiam ...
13	13	ybedboroc	Proin at turpis a ...
14	14	marnautd	Aenean fermentu...
15	15	kjanuaryste	Suspendisse pot...

Total rows: 50 of 50 Query complete 00:00:00.049

19. Make view of players who play football and print this view

create or replace view lists as select *
 from olympics_db.player f1 where f1."player_id"
 in(select t1."player_id" from olympics_db."winner_info" t1
 where t1.sports_name = 'football')
 select * from lists;

Query
Query History

```

32
33 --19. Make view of players who play football and print this view
34 create or replace view lists as select *
35 from olympics_db.player f1 where f1."player_id"
36 in(select t1."player_id" from olympics_db."winner_info" t1
37 where t1.sports_name = 'football')
38 select * from lists;

```

Data output
Messages
Notifications

+

📄

▼

📋

🗑️

🔄

⬇️

📶

	player_id integer	first_name character varying	last_name character varying	gender character varying	height integer	weight integer	dob character varying
1	12	Orren	Grattan	M	181	40	12-08-2013

Total rows: 1 of 1

Query complete 00:00:00.047

20. Create a column 'input_status' in the user_input table. Create a trigger to put the default value as 'Yes' for every new entry if nothing is given by the user. Insert new record to check the functionality of trigger.

```

alter table "olympics_db"."user_input" DROP COLUMN IF EXISTS
update_status;
alter table "olympics_db"."user_input" add column update_status
character varying;

```

```

CREATE OR REPLACE FUNCTION "olympics_db".update_val()
returns trigger
language 'plpgsql'
cost 100
volatile not leakproof
as $body$
begin

```

```

if new.update_status=NULL THEN
update "olympics_db"."user_input" SET update_status='yes' where
NEW."sr_no"="sr_no";
else return new;
end if;
end; $body$;
alter function "olympics_db".update_val() owner to postgres;
create trigger t_ins after insert on "olympics_db"."user_input" for each
row execute procedure "olympics_db".update_val();

```

```

insert into olympics_db."user_input" VALUES('23','NEW','Trigger check');
insert into olympics_db."user_input" VALUES('24','NEW1','Trigger
check1','yes');
insert into olympics_db."user_input" VALUES('24','NEW1','Trigger
check1','no');
select * from "olympics_db"."user_input"

```

Query
Query History
Scratch

```

6 language 'plpgsql'
7 cost 100
8 volatile not leakproof
9 as $body$
10 begin
11 if new.update_status=NULL THEN
12 update "olympics_db"."user_input" SET update_status='yes' where NEW."sr_no"="sr_no";
13 else return new;
14 end if;
15 end; $body$;
16 alter function "olympics_db".update_val() owner to postgres;
17 create trigger t_ins after insert on "olympics_db"."user_input" for each row execute procedure "olympics_db".update_val();
18
19 insert into olympics_db."user_input" VALUES('23','NEW','Trigger check');
20 insert into olympics_db."user_input" VALUES('24','NEW1','Trigger check1','yes');
21 insert into olympics_db."user_input" VALUES('24','NEW1','Trigger check1','no');
22 select * from "olympics_db"."user_input"

```

Data output
Messages
Notifications

	sr_no [PK] integer	type_of_info character varying	info_description character varying	update_status character varying
21	21	NEW	Trigger check	[null]
22	22	NEW1	Trigger check1	[null]
23	23	NEW	Trigger check	[null]
24	24	NEW1	Trigger check1	yes
25	25	NEW2	Trigger check2	no

Total rows: 25 of 25
Query complete 00:00:00.225